Parallel Expectation-Maximization for Clustering

Davide Donà Andrea Blushi

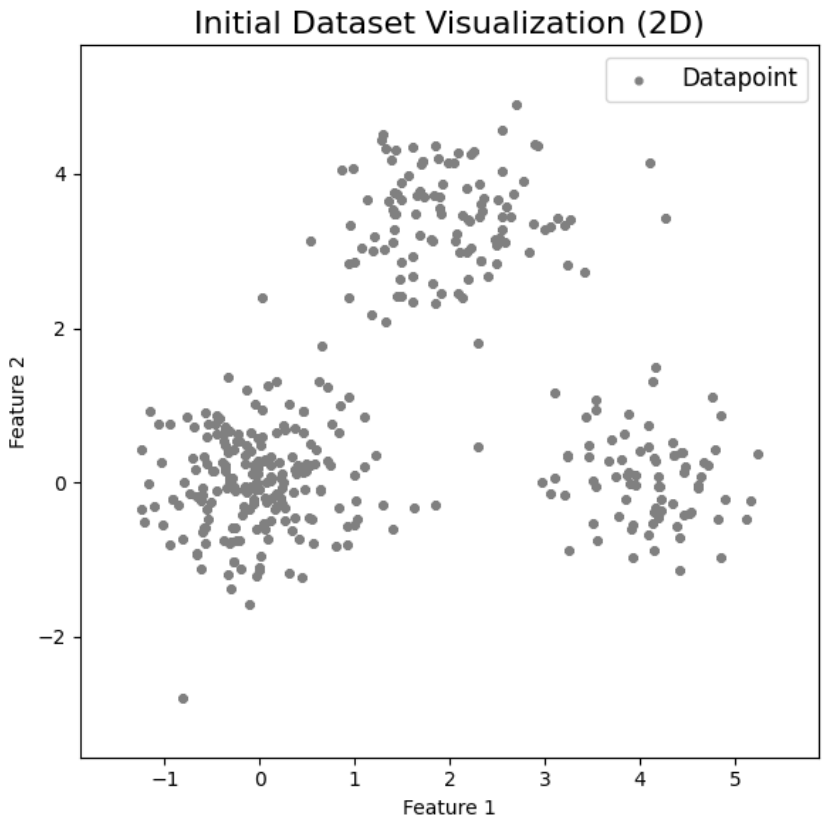
University of Trento

January 2026

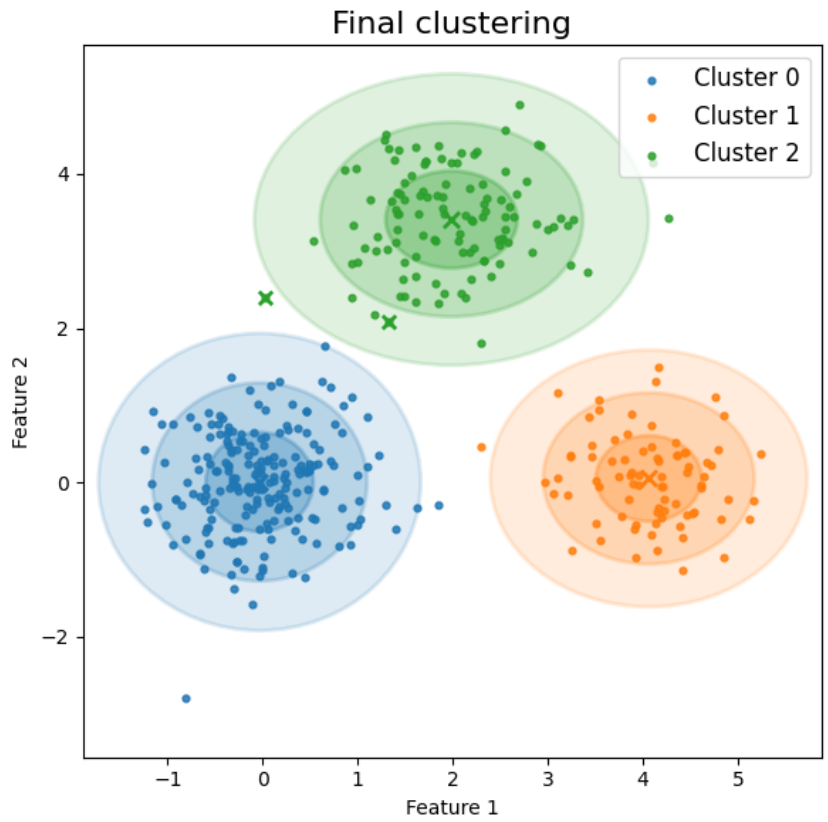
Contents

1. Introduction
2. Parallel Design
3. MPI - Implementation
4. MPI - Scalability
5. Hybrid - Implementation
6. Hybrid - Scalability
7. Conclusion

What is clustering - A Visual Example

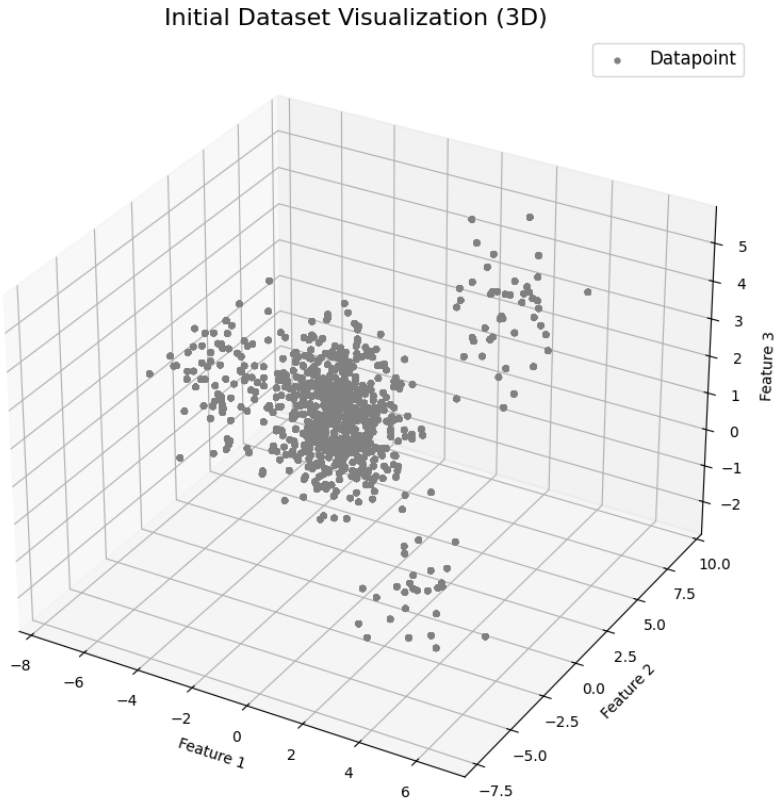


First iteration of the clustering process

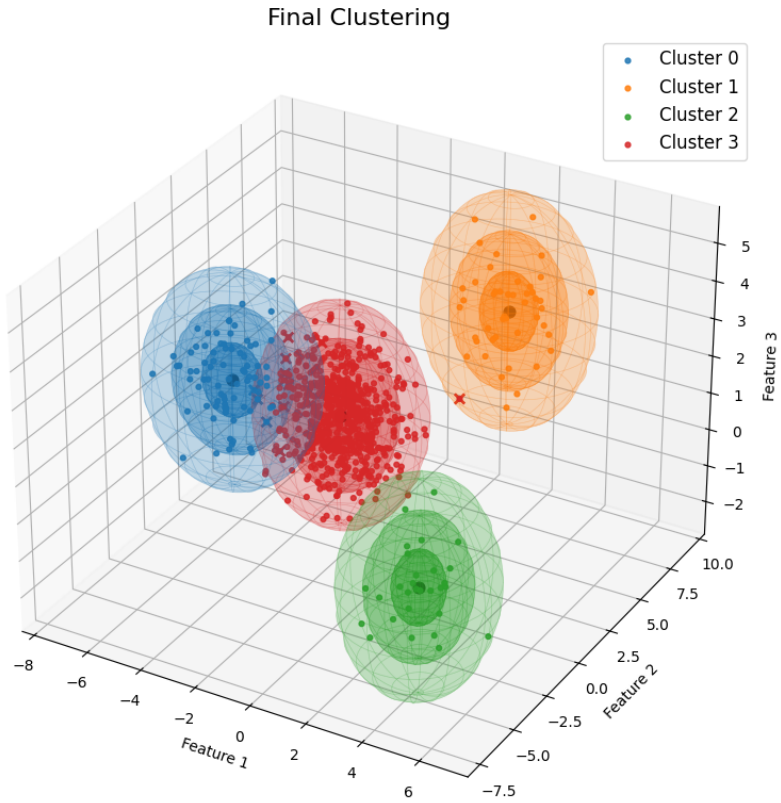


Final clustering result

What is clustering - A Visual Example (3D)



First iteration (3D view)



Final clustering (3D view)

What is clustering - More Formally

- Partition a dataset $\mathcal{X} = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_N\}$ where $\boldsymbol{x}_i \in \mathbb{R}^D$ into K groups (clusters).
- Each cluster is represented by a probability distribution with parameters θ_k.
- Find the cluster parameters that maximize the log-likelihood of the data:

$$\max_{\theta}(\mathcal{L}(\theta|\mathcal{X}))$$

$$\mathcal{L}(\theta|\mathcal{X}) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \, p(\boldsymbol{x}_i \mid \theta_k) \right)$$

Gaussian Mixture Models

- Each cluster is represented by a Gaussian distribution with its own mean and covariance:

$$p(\boldsymbol{x} \mid \theta) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \Sigma_k)$$

$$\sum_{k=1}^{K} \pi_k = 1, \quad \pi_k \geq 0$$

- The likelihood of a data point $\boldsymbol{x}$ to belong to cluster k is given by:

$$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \Sigma_k) = \frac{1}{(2\pi)^{D/2} |\Sigma_k|^{1/2}} \exp\left(-\frac{1}{2} (\boldsymbol{x} - \boldsymbol{\mu}_k)^T \Sigma_k^{-1} (\boldsymbol{x} - \boldsymbol{\mu}_k) \right)$$

Gaussian Mixture Models – Approximation

- We assumed the features are uncorrelated within each component, simplifying the covariance matrix to a diagonal form:

$$\Sigma_k = \begin{pmatrix} \sigma_{k,1} & 0 & \cdots & 0 \\ 0 & \sigma_{k,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_{k,D} \end{pmatrix}$$

- This implies that the likelihood can be expressed as:

$$\mathcal{N}(\boldsymbol{x} \mid \boldsymbol{\mu}_k, \Sigma_k) = \frac{1}{(2\pi)^{D/2} \prod_{d=1}^{D} \sigma_{k,d}^{1/2}} \exp\left(-\frac{1}{2} \sum_{d=1}^{D} \frac{(x_d - \mu_{k,d})^2}{\sigma_{k,d}} \right)$$

Chicken and Egg Problem

- To cluster the data, we need to know the parameters of the Gaussian components.
- To estimate the parameters, we need to know the cluster assignments of the data points.
- This circular dependency is known as the "chicken and egg" problem in clustering.

The solution - EM-Algorithm

The Expectation-Maximization (EM) algorithm is an iterative method to find maximum likelihood estimates of parameters in models with latent variables.

EM-Algorithm - Steps

The EM algorithm consists of two main steps:

- **E-step (Expectation step):** Compute the expected value of the latent variables given the current parameter estimates (soft assignments).
- **M-step (Maximization step):** Update the parameters to maximize the expected log-likelihood found in the E-step.

EM-Algorithm - Pseudocode

Algorithm 1 Expectation-Maximization for GMM clustering

Randomly initialize parameters $\{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$
for i $= 1 \ldots$ MAX_ITER **do**
    E-step: compute responsibilities
    M-step: update parameters
    Compute log-likelihood $\mathcal{L}$
    if Converged **then**
        break
    end if
end for
Clustering: assign labels

EM-Algorithm - E-Step

Using the current parameter estimates θ, we compute the responsibilities, i.e. the posterior probability that data point i belongs to Gaussian component k.

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_j, \Sigma_j)}$$

EM-Algorithm - M-Step

Using the responsibilities γ_{ik} computed in the E-step, we update the parameters:

- **Sum of responsibilities for each component:** $N_k = \sum_{i=1}^{N} \gamma_{ik}$
- **Update of means**: $\boldsymbol{\mu}_k = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} \boldsymbol{x}_i$
- **Update of covariances**: $\Sigma_k = \frac{1}{N_k} \sum_{i=1}^{N} \gamma_{ik} (\boldsymbol{x}_i - \boldsymbol{\mu}_k)(\boldsymbol{x}_i - \boldsymbol{\mu}_k)^T$
- **Update of mixture weights**: $\pi_k = \frac{N_k}{N}$

Convergence

The log-likelihood of the data given the model parameters is computed as:

$$\mathcal{L}(\theta) = \sum_{i=1}^{N} \log \left(\sum_{k=1}^{K} \pi_k \, \mathcal{N}(\boldsymbol{x}_i \mid \boldsymbol{\mu}_k, \Sigma_k) \right)$$

Convergence it's reached if the change is below a predefined threshold ϵ.

Clustering Assignment

After convergence, each data point is assigned to the cluster with the highest responsibility:

$$\hat{z}_i = \arg\max_k p(z_i = k \mid \boldsymbol{x}_i, \theta) = \arg\max_k \gamma_{ik}$$

Computational Complexity Analysis

- The computational cost is expressed in terms of number of data points N, clusters K, and feature dimensions D.
- Both E-step and M-step involve operations over all data points and clusters:

$$O(NKD)$$

- Overall iteration cost:

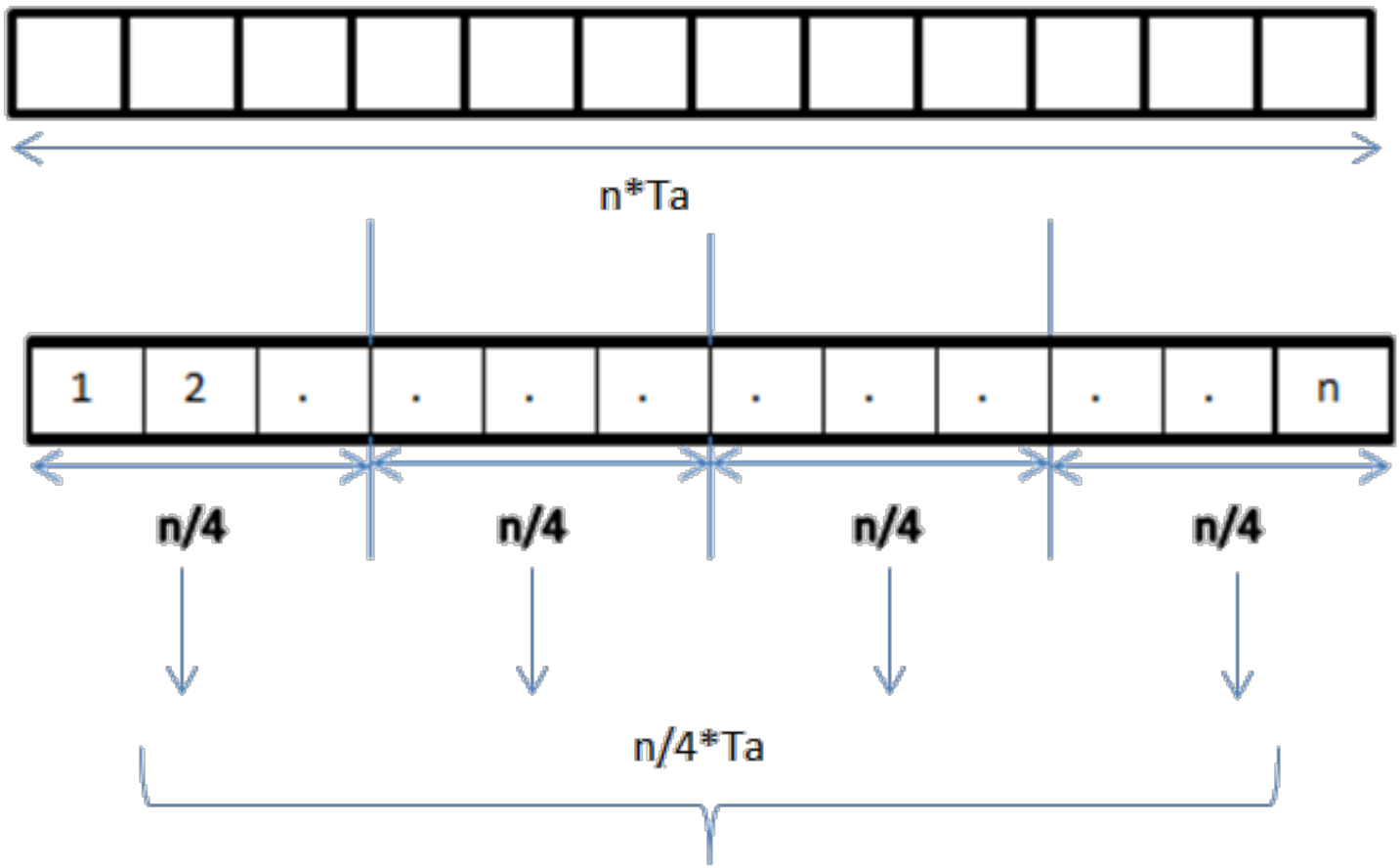
$$O(NKD) + O(NKD) = O(NKD)$$

Parallel Design - Literature Review

State-of-the-Art Approaches

- *Massively parallel expectation maximization using graphics processing units*, M. C. Altinigneli, C. Plant, and C. Bohm
- *A simple parallel em algorithm for statistical learning via mixture models*, S. X. Lee, K. L. Leemaqz, and G. J. McLachlan

Our Approach - Data Parallelization Strategy



Our Approach - Data Distribution

- A master process is responsible for reading the dataset;
- The dataset is then partitioned into P subsets, where P is the number of available workers;
- Each subset is distributed to a different worker process;

Our Approach - E-Step

- Each worker computes the responsibilities for its local subset of data points;
- Given the parameters θ, the responsibilities are computed independently for each data point
- No inter-worker communication is required during this phase.

Our Approach - M-Step

- Each worker computes local sufficient statistics based on the soft assignments calculated in the E-step;
- These local statistics are then aggregated to update the global model parameters;
- Each worker will receive the updated parameters for the next E-step.

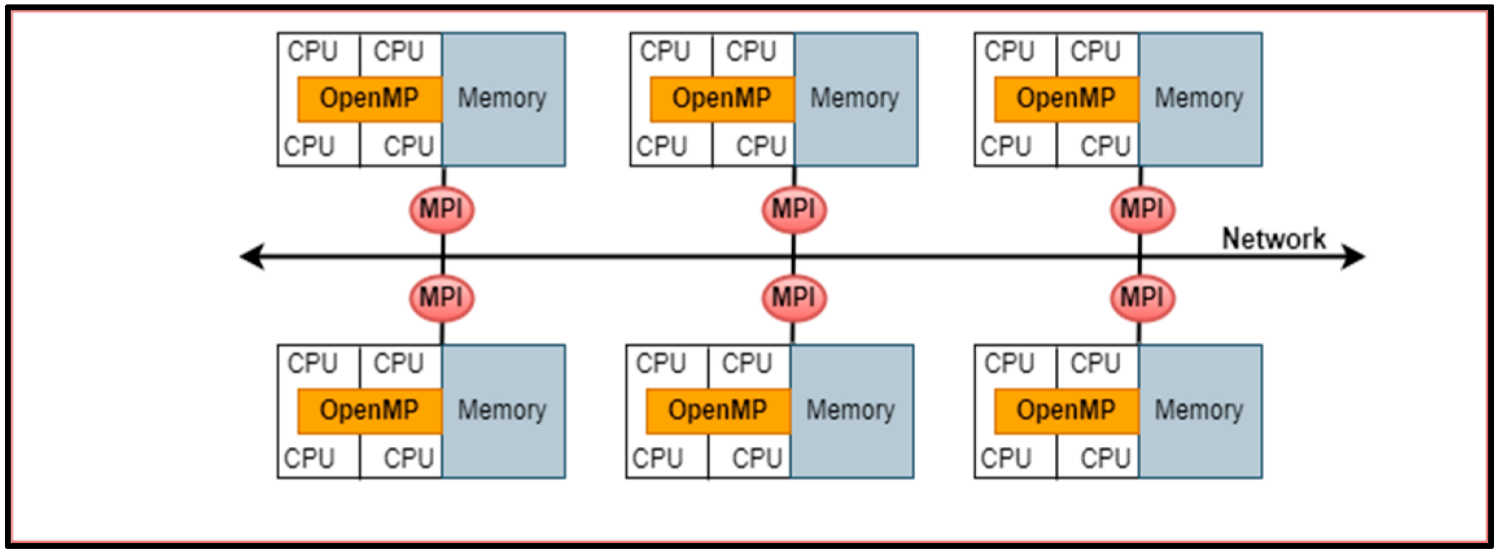
Performance Considerations

- Theoretically, the data parallelization strategy should decrease the complexity to;

$$O\left(\frac{N \cdot K \cdot D}{P}\right)$$

  where P is the number of workers;
- Usually, real-world datasets have $N \gg K$ and $N \gg D$, making data parallelization more effective;
- The communication overhead during the M-step aggregation scales with K and D. This may become a bottleneck with large datasets.

Hybrid Parallelization



Hybrid Parallelization - Theoretical Speedup Analysis

- Reduces the number of MPI processes, lowering the number of communication events and overhead;
- Each MPI Process works on a larger subset of data, allowing computation to occur within a shared-memory environment;

Other Parallelization Strategies

- **Task Parallelization**
- **Pipeline Parallelization**
- **Model Parallelization**

MPI Implementation - Data Distribution

1. The dataset ($\mathcal{X}$), together with the metadata file, is read by the master process (rank 0);
2. The metadata is first collected in an array and then broadcast to all processes using `MPI_Bcast`;
3. Each process calculates the size of its local workload;
4. Using `MPI_Scatterv`, the dataset is partitioned ($\mathcal{X}_{loc}$) and distributed among all processes;
5. After the clustering is complete, all local predicted labels are gathered by the master process using `MPI_Gatherv`.

MPI Implementation - E-Step

- It's inherently data-parallel, requiring only a change to the input parameter of the function.
- Our implementation computes the log-likelihood during this phase.

MPI Implementation- M-Step

- Each worker computes local sufficient statistics based on its local data points.
- These local statistics are then aggregated using `MPI_Allreduce`.
- Each process computes the updated global model parameters for the next E-step.

Scalability - Experimental Setup

- The dataset was synthetically generated via the `make_blobs` function from `sklearn.datasets`.
- Composed of:
  - Various number of samples, from 156k to 10M
  - 50 features
  - 15 clusters
- We ignored the initial data loading and scattering, as they are not part of the EM algorithm.
- We ignored the convergence criterion, fixing the number of iterations to 100.

MPI - Execution Times

N_Processes	0.156M	0.312M	0.625M	1.25M	2.5M	5M	10M
1	281.444	563.894	1027.496	2116.075	4553.415	8741.255	18373.166
2	140.678	261.653	572.785	1013.512	2058.963	4206.922	8887.233
4	71.339	132.982	283.020	522.856	1054.098	2112.717	4437.179
8	34.587	67.079	146.560	280.315	510.900	1100.518	2319.227
16	19.760	34.806	69.208	139.989	264.852	582.296	1153.750
32	9.425	18.796	40.770	75.046	146.378	290.347	601.705
64	5.164	10.042	20.823	41.247	77.118	150.703	321.383

Table: Execution times (in seconds) for different dataset sizes and number of processes.

MPI - Speedup Results

N_Processes	0.156M	0.312M	0.625M	1.25M	2.5M	5M	10M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	2.001	2.155	1.794	2.088	2.212	2.078	2.067
4	3.945	4.240	3.630	4.047	4.320	4.137	4.141
8	8.137	8.406	7.011	7.549	8.913	7.943	7.922
16	14.243	16.201	14.846	15.116	17.192	15.012	15.925
32	29.862	30.001	25.202	28.197	31.107	30.106	30.535
64	54.505	56.154	49.343	51.303	59.045	58.003	57.169

Table: Speedup (T_{serial}/T_p) for different dataset sizes and number of processes p.

MPI - Speedup Results

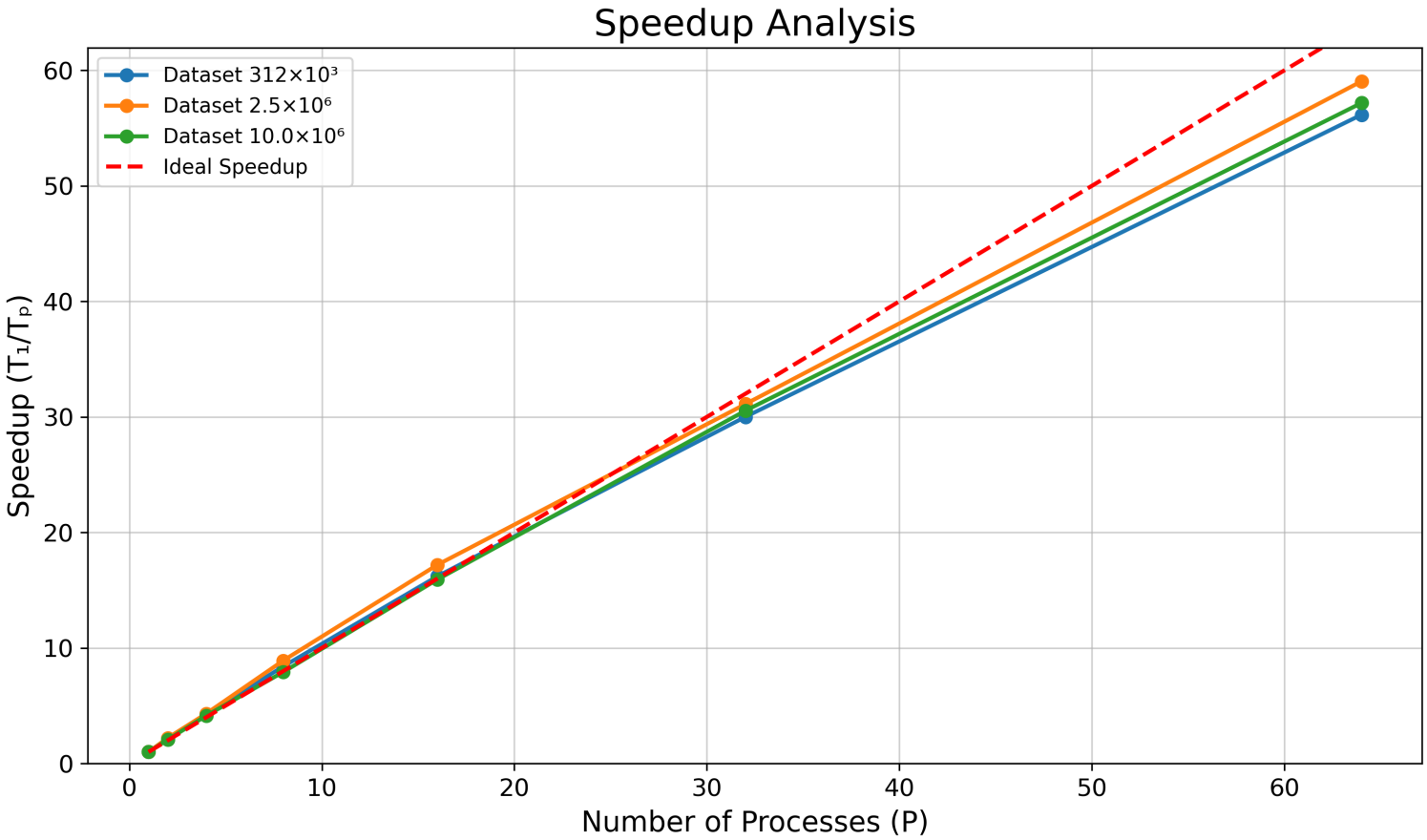


Figure: Speedup plot for different dataset sizes (small, medium, large). The dashed line represents the ideal linear speedup.

MPI - Efficiency Results

N_Processes	0.156M	0.312M	0.625M	1.25M	2.5M	5M	10M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.000	1.078	0.897	1.044	1.106	1.039	1.034
4	0.986	1.060	0.908	1.012	1.080	1.034	1.035
8	1.017	1.051	0.876	0.944	1.114	0.993	0.990
16	0.890	1.013	0.928	0.945	1.075	0.938	0.995
32	0.933	0.938	0.788	0.881	0.972	0.941	0.954
64	0.852	0.877	0.771	0.802	0.923	0.906	0.893

Table: Efficiency (Speedup / p) for different dataset sizes and number of processes.

MPI - Efficiency Results

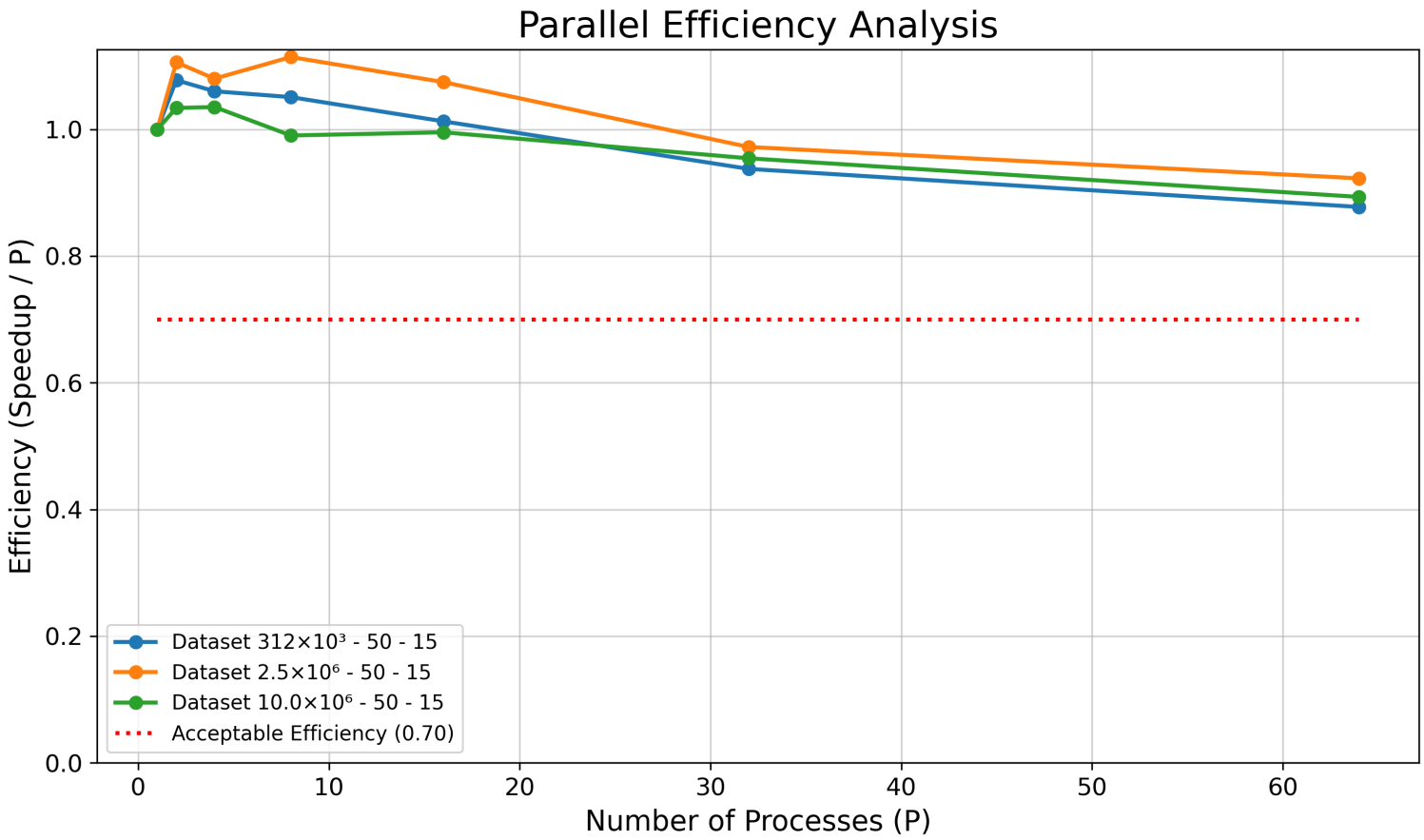


Figure: Efficiency plot for different dataset sizes (small, medium, large). The dashed line indicates the target efficiency of 0.70.

MPI - E-Step vs M-Step Scalability

NP	0.156	0.312	0.625	1.25	2.5	5	10
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.003	1.084	0.898	1.045	1.106	1.042	1.035
4	0.996	1.070	0.918	1.016	1.081	1.041	1.033
8	1.028	1.081	0.910	0.967	1.142	1.050	1.039
16	0.921	1.031	0.943	0.969	1.093	0.949	1.028
32	0.956	0.962	0.830	0.910	1.008	0.963	0.981
64	0.883	0.925	0.821	0.849	0.961	0.944	0.923

Table: E-Step efficiency for different dataset sizes and number of processes.

NP	0.156	0.312	0.625	1.25	2.5	5	10
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	0.963	0.993	0.875	1.035	1.107	0.995	1.022
4	0.857	0.924	0.773	0.947	1.064	0.949	1.072
8	0.866	0.738	0.557	0.691	0.826	0.549	0.601
16	0.576	0.793	0.745	0.685	0.866	0.800	0.690
32	0.673	0.673	0.441	0.594	0.644	0.701	0.695
64	0.543	0.490	0.398	0.437	0.584	0.568	0.617

Table: M-Step efficiency for different dataset sizes and number of processes.

MPI - A more in depth look

To better understand the scalability problem within the M-Step, we decided to run additional experiments with a different datasets.
In particular, we increased the number of features $D = 300$ and clusters $K = 20$.

MPI - A more in depth look

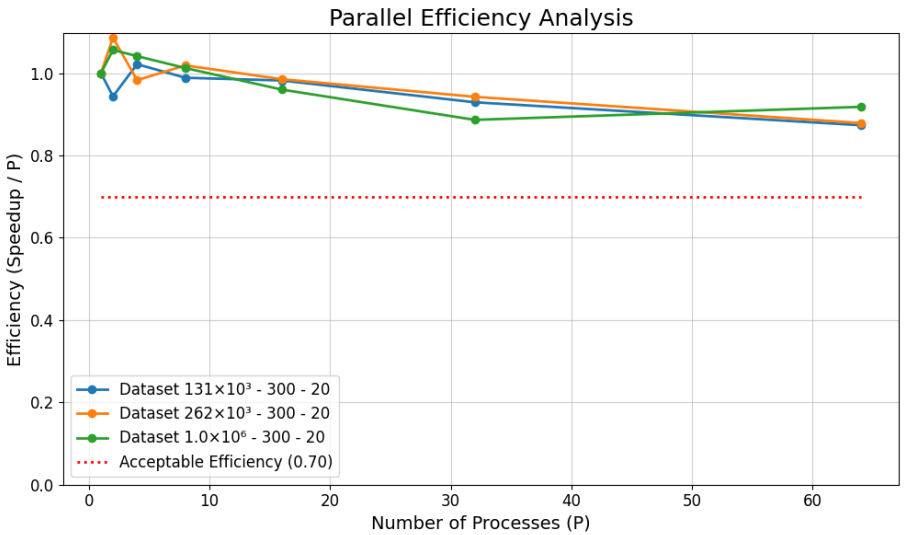


Figure: Efficiency plot for different dataset sizes (small, medium, large) with increased features and clusters. The dashed line indicates the target efficiency of 0.70.

NP	0.033	0.066	0.131	0.262	0.524	1.049	2.097
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.026	1.074	0.694	1.037	1.024	1.021	0.820
4	1.010	0.901	0.817	0.632	0.633	1.002	0.752
8	0.749	0.636	0.672	0.840	0.662	0.674	0.589
16	0.523	0.510	0.806	0.693	0.555	0.767	0.788
32	0.621	0.581	0.538	0.654	0.540	0.415	0.490
64	0.322	0.324	0.468	0.452	0.408	0.569	0.292

Table: M-Step efficiency for different dataset sizes and number of processes.

Hybrid - Implementation

- The hybrid implementation was built upon the existing MPI implementation by integrating `OpenMP` directives.
- A **static scheduling** strategy was chosen, as the workload is evenly distributed among the threads.

Hybrid Implementation - E-Step

- We used *#pragma omp parallel* to parallelize the outer loop that iterates over data points.
- No relevant data dependencies exist between iterations; only the log-likelihood requires reduction since has a flow loop-carried data dependency.

Hybrid Implementation - M-Step

1. All sufficient statistics have a flow loop-carried data dependency.
2. `OpenMP` directives were used to parallelize the computation of local sufficient statistics.
3. A private copy of the local statistics was created for each thread, which was then reduced to obtain the final local statistics.
4. During the aggregation of N_k and u_{kd}, it was possible to use *#pragma omp nowait*.
5. After all thread-private statistics were aggregated, `MPI_Allreduce` was used to compute the global statistics.
6. Each process then computed the updated global model parameters for the next E-step.

Hybrid - Efficiency Results

N_Processes	0.033M	0.066M	0.131M	0.262M	0.524M	1.049M	2.097M
1	1.000	1.000	1.000	1.000	1.000	1.000	1.000
2	1.040	0.975	0.984	0.985	0.968	1.029	0.840
4	1.075	1.057	0.970	1.068	0.955	1.052	0.929
8	1.055	0.981	0.978	1.000	0.996	1.038	0.907
16	1.003	0.929	0.880	0.893	0.915	0.923	0.860
32	0.974	0.872	0.828	0.970	0.888	0.903	0.822
64	0.925	0.862	0.854	0.922	0.912	0.897	0.868

Table: Efficiency for Hybrid approach for different dataset sizes and number of processes.

Hybrid - Efficiency Results

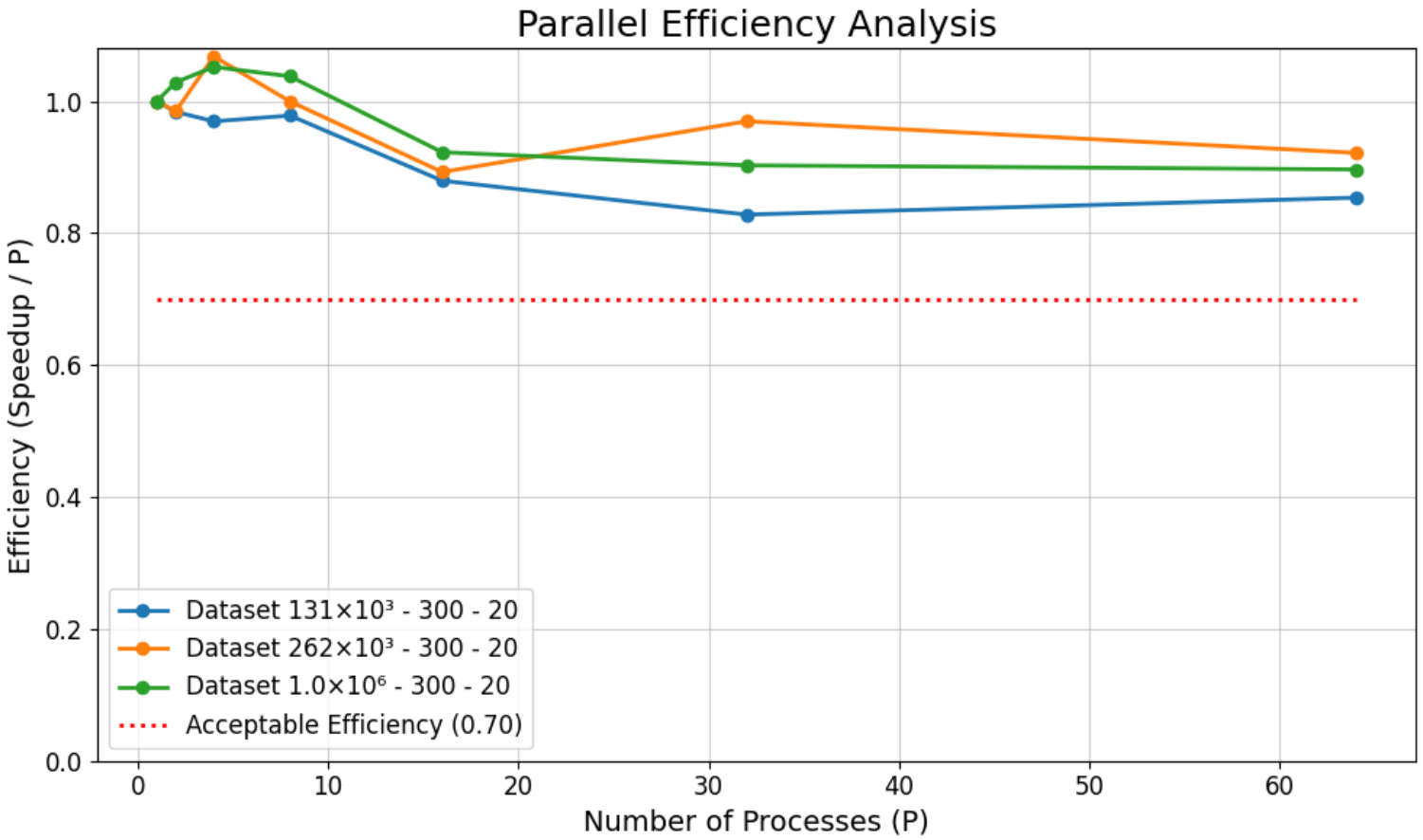


Figure: Hybrid Efficiency plot for different dataset sizes (small, medium, large). The dashed line indicates the target efficiency of 0.70.

Hybrid - A more in depth look

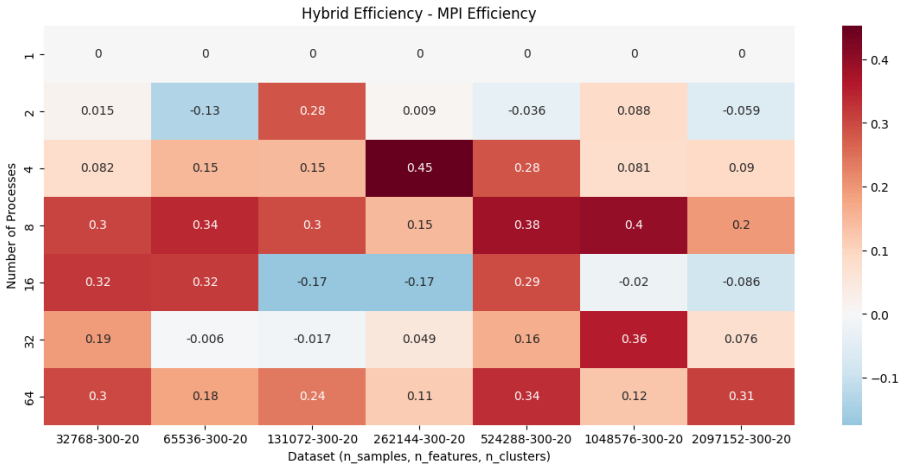


Figure: M-Step Efficiency Heatmap for difference between Pure MPI and Hybrid approach. Red indicates higher efficiency in Hybrid, blue indicates higher efficiency in Pure MPI.

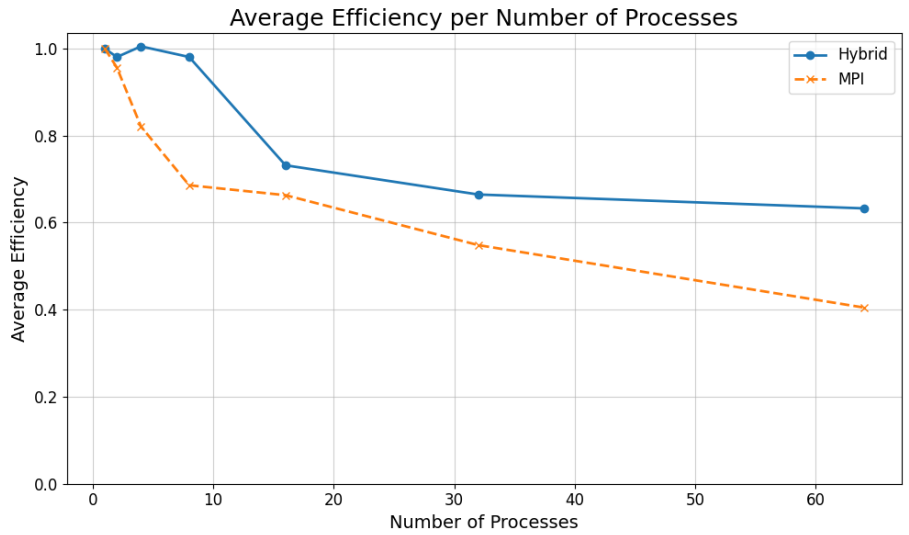


Figure: Average M-Step Efficiency Comparison between Pure MPI and Hybrid approach.

Conclusion

- Both the MPI and hybrid implementations exhibit excellent efficiency and weak scalability.
- As K and D increase, communication overhead becomes more significant, negatively impacting MPI performance.
- The hybrid approach effectively utilizes multi-core architectures, reducing communication overhead.
- Performance is consistent with the state-of-the-art literature.

Thank you!

[1] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the em algorithm," *Journal of the Royal Statistical Society: Series B*, vol. 39, no. 1, pp. 1–38, 1977.

[2] D. Reynolds, *Gaussian Mixture Models*. Boston, MA: Springer US, 2009.

[3] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[4] A. Kak, "Expectation–maximization algorithm for clustering multidimensional numerical data: An rvl tutorial," Tech. Rep. —, Purdue University, March 3 2024.

[5] M. C. Altinigneli, C. Plant, and C. Böhm, "Massively parallel expectation maximization using graphics processing units," in *Proceedings of the 19th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, KDD '13, (New York, NY, USA), p. 838–846, Association for Computing Machinery, 2013.

[6] S. X. Lee, K. L. Leemaqz, and G. J. McLachlan, "A simple parallel em algorithm for statistical learning via mixture models," in *2016 International Conference on Digital Image Computing: Techniques and Applications (DICTA)*, pp. 1–8, 2016.

[7] I. Azizi, "Parallelization in python - an expectation-maximization application," tech. rep., Université de Lausanne, Département des Opérations (DO), June 2023. Inspired by CUSO Informatique 2023 Winter School.